

1ER PARCIAL

Nahuel Alejandro Acevedo
ISTEA Minería de datos 2

1) Interpretación del problema, usuarios, preguntas principales y objetivos medibles:

La situación es la de un equipo en el área de datos con la tarea de ingestar, limpiar, conformar y publicar datos de clientes para su análisis. El problema que surge a partir de esto es la existencia de datos crudos con inconsistencias, nulos, tipos ambiguos, anomalías y la evolución del esquema durante el proceso.

Los usuarios identificados para este conjunto de sets de datos son finOps, para el análisis de aspectos netamente contables y económicos; Soporte, para el cálculo del volumen de los tickets y verificaciones; y Producto/Usage para el análisis del uso de servicios, requests y desarrollo de métricas operativas.

Se determinan como preguntas principales que deberán responderse analizando este conjunto de sets de datos las siguientes: los costos y requests diarios por organización y servicio en un rango de fechas, los servicios con mayor costo acumulado en los últimos catorce días, evolución de tickets críticos y tasa de SLA breach por día durante los últimos treinta días, ingreso mensual con créditos e impuestos aplicados y medido en dólares, tokens GenAI y costo estimado diario.

Se establecen como objetivos medibles la determinación de un pipeline utilizando PySpark para la lectura de los datasets y luego su guardado utilizando Parquet, la generación de un Data Lake diferenciando correctamente en las zonas Bronze, Silver y Gold, y por último garantizar la posibilidad de utilización y acceso a los datos necesarios con la base de datos no relacional Cassandra.

2) Justificación de la necesidad de Big Data mediante volumen, velocidad, variedad, veracidad y valor:

Volumen: existen múltiples archivos con miles de registros, por lo que con el desarrollo del negocio el procesamiento utilizando una sola computadora será inviable.

Velocidad: existen no sólo datos estáticos (batch) sino también casi de tiempo real (streaming) con el uso de archivos jsonl, por lo que la velocidad de procesamiento no podría desarrollarse sin capacidades de streaming, las cuales Spark abarca satisfactoriamente.

Variedad: hay dos formatos distintos de archivos, CSV (con datos estructurados) y JSONL (con datos no estructurados), además de que se desarrolla una evolución de

esquemas mediante la incorporación de los campos carbon_kg y genai_tokens en la segunda versión de los eventos. Ambos aspectos no podrían ser tratados mediante el uso de una base de datos estructurada.

Veracidad: hay una gran cantidad de inconsistencias y datos nulos, por lo que los datos crudos cuentan con una baja veracidad, ante lo cual el uso de Big Data permite generar zonas de cuarentena y procesamiento masivo para la limpieza satisfactoria y eficiente de estos datos.

Valor: la mayor eficiencia en el tratamiento esta enorme cantidad de datos al utilizar Big Data permite a los usuarios interesados desarrollar análisis mucho más útiles para la toma de decisiones en la empresa.

3) Inventario y perfil inicial de las fuentes: grano, frecuencia, tipos, calidad, trazabilidad y riesgos:

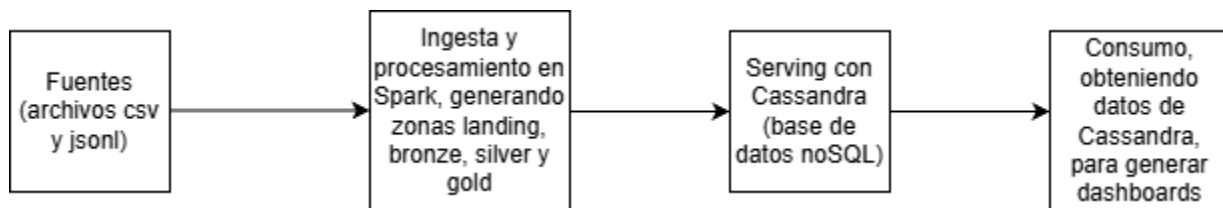
Fuente	Granularidad	Frecuencia	Tipo de dato	Calidad
billin_monthly.csv	una factura mensual por organización	Batch	Estructurado	Muy baja, con 137 nulos en credits y 13 negativos en subtotal
support_tickets.csv	un ticket de soporte	Batch	Estructurado	Baja, con 240 nulos en solved_at y 254 nulos en csat
customers_orgs.csv	Un cliente/organización	Batch	Estructurado	Media, con 11 nulos en nps_score
users.csv	Un usuario en una organización	Batch	Estructurado	Media, con 139 nulos repartidos en distintos campos
resources.csv	Un recurso cloud provisto	Batch	Estructurado	Media, con 83 nulos repartidos

nps_surveys.csv	Una encuesta de satisfacción temporal	Batch	Estructurado	Media, con 29 valores nulos
marketing_touches.csv	Una interacción de marketing	Batch	Estructurado	Excelente, sin datos nulos
usage_events_stream/*.jsonl	Un evento de uso/consumo	Streaming	Semi-estructurado	Buena, con archivos fragmentados intencionalmente

Respecto a la trazabilidad, se agregarán las columnas `ingest_ts` para el momento exacto del procesamiento y `source_file` con la ruta del archivo de origen para determinar de dónde provino cada fila.

Respecto a los riesgos, se encuentra aquel acerca de la evolución del esquema, en la que la segunda versión de los logs de streaming incorporan dos campos nuevos `carbon_kg` y `genai_tokens`; así como también existe el riesgo respecto de tipos de datos ambiguos, por números ingresados como texto y valores numéricos negativos que no deberían serlo.

4) Diagrama de arquitectura de alto nivel con fuentes, ingesta, Data Lake, procesamiento, serving, consumo y capacidades transversales:



5) Selección justificada del patrón batch, Lambda, Kappa o híbrido:

Se elige el patrón Lambda, ya que es necesario contar con capacidades de batch y de streaming para los archivos csv y los archivos jsonl respectivamente en este proyecto, ya que los archivos csv cuentan con datos de carácter histórico, mientras que los jsonl cuentan con datos casi en tiempo real.

6) Mapeo de requisitos a componentes, incluyendo la relación entre las 5V y las decisiones de arquitectura:

Volumen: El data lake permite manipular la gran cantidad de datos propios del desarrollo del negocio de manera eficiente.

Variedad: La diversidad en los formatos de los archivos y la evolución dada en ellos son correctamente tratados con el data lake.

Velocidad: El streaming con Spark desarrolla correctamente el uso de los datos en casi tiempo real de los archivos JSONL.

Veracidad: El uso de PySpark permite un tratamiento y procesamiento eficientes respecto a los nulos y las anomalías en los datos.

Valor: La utilización de Cassandra en la etapa de serving otorga el acceso eficiente a los datos para llevar a cabo las consultas de orden analítico.

7) Diseño del Data Lake: zonas, formatos, particiones, naming, retención, metadatos y reglas de promoción:

Se diseña el Data Lake a partir de las zonas Bronze/Silver/Gold (estructura de medallón), luego de la zona inicial de Landing (datos inmutables tal cual como se obtienen de las fuentes)

Landing: almacenamiento de los archivos originales CSV y JSONL sólo para lectura. La regla de promoción es la de su pasaje con lectura íntegra a la zona Bronze.

Bronze: mantención de los datos de la etapa anterior, pero con incorporación de campos para trazabilidad (en este caso ingest_ts para el tiempo con ts significando time stamp y source_file para la indicación de la fuente). La regla de promoción es que los datos serán procesados luego en la etapa Silver.

Silver: limpieza, estandarización y normalización, tratando los valores nulos y filtrando los negativos inconsistentes, así como también estandarizando los tipos de

datos inconsistentes. La regla de promoción es que estarán los datos listos para joins y consultas según las reglas de negocio en la siguiente etapa Gold.

Gold: modelación de tablas para el uso analítico de los datos.

Respecto al formato, se determina el uso de Apache Parquet desde la etapa Bronze por su mayor eficiencia para análisis.

Partición: se llevarán a cabo particiones de acuerdo a aspectos lógicos como fecha (año, mes) o servicio para llevar a cabo la poda de particiones optimizando así las lecturas de los datos.

Naming: se conservarán tanto la convención snake_case como el idioma inglés.

Retención y metadatos: en la zona Landing se hará una retención temporal mientras que desde Bronze hasta Gold se guardarán los datos históricos. Se van a determinar esquemas explícitos para la evolución del esquema en los archivos JSONL.

8) Flujo de datos batch y streaming desde las fuentes hasta las salidas, con herramientas específicas:

Flujo Batch: las fuentes serán los archivos CSV (datos históricos), utilizándose spark.read.csv de PySpark (para dataframes), yendo desde Landing, pasando por Bronze, Silver y Gold para limpieza y agregación, garantizándose la idempotencia con por ejemplo mode("overwrite"), para luego mediante Cassandra (noSql) se habilitará el consumo de datos de manera eficiente.

Flujo Streaming: archivo de eventos continuos usage_events_stream/*.jsonl, utilizándose Spark Structured Streaming (spark.readStream), para finalmente insertar los datos en Cassandra.

9) Flujo batch de referencia expresado con lógica MapReduce o equivalente conceptual:

MAP (narrow): los nodos ejecutores de Spark leen los archivos de manera independiente y en paralelo mediante operaciones como .select() o .filter().

Shuffle (intercambio en red): redistribución de los datos a través de la red del clúster, de manera que todos los valores agrupados mediante la misma clave se agrupan en la misma partición para operarse en conjunto.

Reduce(wide): el nodo ejecutor aplica una agregación final (funciones como `.groupBy().agg(sum())`) sobre los valores.

10) Supuestos, riesgos iniciales, mitigaciones y decisiones todavía abiertas:

Como supuestos contamos con que a excepción de la versión de para los eventos (JSONL) las estructuras de los archivos CSV serán estables.

Existe un riesgo de rotura de la lectura del pipeline mediante la incorporación de los dos campos nuevos en el streaming, pero lo cual será mitigado con el uso de esquemas explícitos.

Hay también riesgos con la veracidad de los datos, pero serán mitigados con un procesamiento correcto de los nulos y los negativos inconsistentes en la zona Silver.

La decisión abierta es la determinación específica del algoritmo utilizado.

11) Estimación preliminar de esfuerzo, roles y recursos requeridos:

En principio se estima un esfuerzo dado en un tiempo de entre cuatro y seis semanas, pero podría modificarse según cómo vaya tratándose específicamente cada etapa.

Los roles requeridos serán los de ingeniero de datos para el desarrollo del pipeline y la estructuración del data lake, el arquitecto de datos para generar las particiones y el desarrollador de calidad para el establecimiento de las reglas de cuarentena.

Los recursos técnicos son: un entorno de ejecución de PySpark (Google colab por ejemplo), una base de datos Cassandra y un repositorio de control de versiones en GitHub.